<u>Internship Task Report</u>

C/C++ Programming Internship

Submitted By:

Nafeez Sultana

College: GITAM University

Branch:

Computer Science and Engineering

Internship Role: C++ / C Programming Intern

Date: May 2026

Table of Contents

1. Introduction
2. Beginner Level Tasks
3. Task 1 — Data Structures
4. Task 2 — Algorithms & Problem Solving
5. Task 4 — C++ OOP Mini Project
6. Conclusion

Introduction

The C/C++ Programming Internship was undertaken with the objective of strengthening foundational programming knowledge and improving practical problem-solving skills using the C and C++ programming languages. This internship provided an opportunity to understand core programming concepts such as data structures, algorithms, object-oriented programming, file handling, memory management, and debugging techniques. The tasks assigned during the internship were designed to enhance logical thinking, coding practices, and the ability to develop efficient and structured programs.

Throughout the internship, various beginner-level and project-based tasks were implemented using tools and technologies such as GCC/G++, GitHub, Makefile, and standard debugging utilities. The internship focused on developing clean, readable, and reusable code while also encouraging proper documentation and testing practices. Practical implementation of concepts such as stacks, queues, linked lists, sorting algorithms, searching techniques, and object-oriented applications helped in gaining hands-on programming experience.

In addition to coding tasks, the internship also introduced important software development practices such as compiling and executing programs through command-line tools, maintaining project files, organizing code repositories using GitHub, and documenting outputs with screenshots and explanations. These activities helped in understanding the complete workflow involved in software development.

Beginner Level Task 1 — Fibonacci Series

Aim

To implement Fibonacci series using iterative and recursive methods in C.

Tools Used

- GCC Compiler
- OnlineGDB / Programiz
- GitHub
-

```c
#include <stdio.h>

// Recursive function
int fibonacci(int n)
{
    if (n == 0)
        return 0;
    else if (n == 1)
        return 1;
    else
        return fibonacci(n - 1) + fibonacci(n - 2);
}

int main()
{
    int n, i;
    int first = 0, second = 1, next;

    printf("Enter number of terms: ");
    scanf("%d", &n);

    // Iterative Approach
    printf("\nFibonacci Series using Iterative Approach:\n");

    for(i = 0; i < n; i++)
    {
        if(i <= 1)
            next = i;
```

-
```c
    // Iterative Approach
    printf("\nFibonacci Series using Iterative Approach:\n");

    for(i = 0; i < n; i++)
    {
        if(i <= 1)
            next = i;
        else
        {
            next = first + second;
            first = second;
            second = next;
        }

        printf("%d ", next);
    }

    // Recursive Approach
    printf("\n\nFibonacci Series using Recursive Approach:\n");

    for(i = 0; i < n; i++)
    {
        printf("%d ", fibonacci(i));
    }

    return 0;
}
```

OUTPUT:

```
Enter number of terms: 4

Fibonacci Series using Iterative Approach:
0 1 1 2

Fibonacci Series using Recursive Approach:
0 1 1 2
```

Beginner Level Task 2: Create a C class that models a Bank Account (deposit, withdraw, display balance).

Aim

To develop a C program that models a Bank Account using Object-Oriented Programming concepts and performs basic banking operations such as deposit, withdrawal, and balance display.

Code:

Run Debug Stop Share Save { } Beautify

main.c

```c
#include <stdio.h>
#include <string.h>

struct BankAccount
{
    int accountNumber;
    char name[50];
    float balance;
};

// Deposit function
void deposit(struct BankAccount *acc, float amount)
{
    acc->balance += amount;
    printf("Amount Deposited: %.2f\n", amount);
}

// Withdraw function
void withdraw(struct BankAccount *acc, float amount)
{
    if(amount <= acc->balance)
    {
        acc->balance -= amount;
        printf("Amount Withdrawn: %.2f\n", amount);
    }
    else
    {
        printf("Insufficient Balance!\n");
```

main.c

```c
}

// Display balance function
void displayBalance(struct BankAccount acc)
{
    printf("\nAccount Number: %d\n", acc.accountNumber);
    printf("Account Holder: %s\n", acc.name);
    printf("Current Balance: %.2f\n", acc.balance);
}

int main()
{
    struct BankAccount account;

    account.accountNumber = 101;
    strcpy(account.name, "Nafeez");
    account.balance = 5000;

    displayBalance(account);

    deposit(&account, 2000);

    withdraw(&account, 1500);

    displayBalance(account);

    return 0;
}
```

OUTPUT:

```
Account Number: 101
Account Holder: Nafeez
Current Balance: 5000.00
Amount Deposited: 2000.00
Amount Withdrawn: 1500.00

Account Number: 101
```

```
Amount Withdrawn: 1500.00

Account Number: 101
Account Holder: Nafeez
Current Balance: 5500.00
```

TASK 1 — Data Structures Implementation

Objective

The objective of this task is to implement core data structures such as Stack and Queue using the C programming language. The task demonstrates basic operations like insertion, deletion, and display along with understanding of time complexity and data organization.

Data Structures Implemented

1. Stack
2. Queue

---

1. Stack Implementation

Aim

To implement Stack data structure using arrays in C and perform push, pop, and display operations.

Algorithm

Push Operation

1. Check if stack is full.
2. Increment top.
3. Insert element into stack.

Pop Operation

1. Check if stack is empty.
2. Remove top element.
3. Decrement top.

Display Operation

1. Print all stack elements from top to bottom.

Run Debug Stop Share Saved { } Beautify

main.c

```c
#include <stdio.h>

#define MAX 5

int stack[MAX];
int top = -1;

// Push Operation
// Time Complexity: O(1)
void push(int value)
{
    if(top == MAX - 1)
    {
        printf("Stack Overflow\n");
    }
    else
    {
        top++;
        stack[top] = value;
        printf("%d pushed into stack\n", value);
    }
}

// Pop Operation
// Time Complexity: O(1)
void pop()
{
    if(top == -1)
```

main.c

```c
{
    if(top == -1)
    {
        printf("Stack Underflow\n");
    }
    else
    {
        printf("%d popped from stack\n", stack[top]);
        top--;
    }
}

// Display Operation
// Time Complexity: O(n)
void display()
{
    int i;

    if(top == -1)
    {
        printf("Stack is empty\n");
    }
    else
    {
        printf("Stack Elements:\n");

        for(i = top; i >= 0; i--)
        {
```

main.c

```c
    }
    else
    {
        printf("Stack Elements:\n");

        for(i = top; i >= 0; i--)
        {
            printf("%d\n", stack[i]);
        }
    }
}

int main()
{
    push(10);
    push(20);
    push(30);

    display();

    pop();

    display();

    return 0;
}
```

```
30 popped from stack
Stack Elements:
20
10

...Program finished with exit code 0
Press ENTER to exit console.
```

Explanation:

The stack follows the LIFO (Last In First Out) principle. Elements are inserted using push operation and removed using pop operation. The display function shows all elements currently present in the stack.

2. Queue Implementation

Aim

To implement Queue data structure using arrays in C and perform enqueue, dequeue, and display operations.

Algorithm

Enqueue Operation

1. Check if queue is full.
2. Insert element at rear position.

Dequeue Operation

1. Check if queue is empty.
2. Remove element from front position.

Display Operation

1. Print all queue elements from front to rear.

Run Debug Stop Share Save { } Beautify

main.c

```c
#include <stdio.h>

#define MAX 5

int queue[MAX];
int front = -1;
int rear = -1;

// Enqueue Operation
// Time Complexity: O(1)
void enqueue(int value)
{
    if(rear == MAX - 1)
    {
        printf("Queue Overflow\n");
    }
    else
    {
        if(front == -1)
        {
            front = 0;
        }

        rear++;
        queue[rear] = value;

        printf("%d inserted into queue\n", value);
```

Run Debug Stop Share Save { } Beautify

main.c

```c
    }
}

// Dequeue Operation
// Time Complexity: O(1)
void dequeue()
{
    if(front == -1 || front > rear)
    {
        printf("Queue Underflow\n");
    }
    else
    {
        printf("%d removed from queue\n", queue[front]);
        front++;
    }
}

// Display Operation
// Time Complexity: O(n)
void display()
{
    int i;

    if(front == -1 || front > rear)
    {
        printf("Queue is empty\n");
```

```c
        printf("Queue is empty\n");
    }
    else
    {
        printf("Queue Elements:\n");

        for(i = front; i <= rear; i++)
        {
            printf("%d\n", queue[i]);
        }
    }
}

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);

    display();

    dequeue();

    display();

    return 0;
}
```

OUTPUT:

```
10 removed from queue
Queue Elements:
20
30

...Program finished with exit code 0
Press ENTER to exit console.
```

Explanation

The queue follows the FIFO (First In First Out) principle. Elements are inserted from the rear using enqueue operation and removed from the front using dequeue operation.

Conclusion

In this task, Stack and Queue data structures were successfully implemented using the C programming language. Basic operations such as insertion, deletion, and display were demonstrated successfully. This task improved understanding of data organization, memory handling, and implementation of linear data structures.